

TFM Failure Cases — Project Summary

Where TabPFN-3, TabICL v2, and TabPFN-2.5 break down on wide, short tabular data — and what that breakdown actually looks like.

Course: AutoML #4 · Supervisors: Omar, Alex

Final submission (code + poster): 05.08.2026 · Poster session: 12.08.2026, 10:00–12:00

This document consolidates the full project as it stands: the original hypothesis, the main sklearn-based failure-case study (complete), the second arithmetic-chain study built to close gaps versus the assignment brief (Experiment 1 complete, Experiments 2 and 3 defined but not yet run), the XGBoost classical-ML baseline added for the poster, and everything currently known, limited, or still open.

1. Hypothesis

On very wide, very short tabular data (few rows, thousands of columns — e.g. 100 rows × 5,000 columns), TabPFN-3 and TabICL v2 degrade sharply relative to TabPFN-2.5, and the reason is architectural, not just “transformers don't scale to many features.”

Why this is an architecture story, not just a scaling limit

Each model's own documentation gives away a different design ceiling on feature count:

Model	Max rows	Max features	Source
TabPFN-2.5	100,000	2,000	Prior Labs docs
TabPFN-3	1,000,000*	200*	Prior Labs changelog
TabICL v2	—	~100**	TabICL PyPI page

* TabPFN-3 trades rows against features rather than having one fixed ceiling: $1\text{M} \times 200$, or $100\text{K} \times 2\text{K}$, or $1\text{K} \times 20\text{K}$, depending on configuration.

** TabICL v2's pretraining distribution covers 2–100 columns; it claims, but doesn't guarantee, generalization beyond that.

- **TabPFN-3** was optimized to scale to a million *rows* — its default row/feature trade-off spends that budget on rows, not columns, unless explicitly configured otherwise.
- **TabICL v2's** entire pretraining distribution tops out at 100 columns; 5,000 is 50x outside anything it was trained on.
- **TabPFN-2.5**, by contrast, was specifically built for width — validated up to 2,000 features.

There's also a concrete mechanism, not just a size limit. TabPFN's own source (`tabpfn/constants.py`) defines `AUTO_FEATURE_SUBSAMPLING_TOP_K = 150` with a trigger threshold of 200 features. Above that threshold, TabPFN doesn't attend over all columns at once — each ensemble member gets a random/balanced subset of features, and predictions are combined across the ensemble. If informative signal is spread across hundreds of columns, each ensemble member works with an incomplete, randomly chosen picture, and accuracy should degrade as the informative-to-total column ratio shrinks with scale. That's the mechanism the main study is built to expose.

TabICL v2 has a related but distinct issue: its complexity is $O(n^2 + nm^2)$ in rows (n) and columns (m), and its synthetic pretraining prior was never sampled with more than 100 columns — its per-column attention patterns are being asked to extrapolate into a completely unseen regime.

2. Main Study — Method

`data_generation.py` builds synthetic 100-row datasets sweeping column count (50 → 5,000, log-spaced) for both classification and regression, using scikit-learn's `make_classification / make_regression`. The number of informative columns scales with total column count (10% of columns, capped 8–300) rather than staying fixed — deliberately, so that per-estimator feature subsampling is actually stressed as the dataset widens, instead of almost any random column slice still catching a real signal column.

Noise composition of the synthetic data

Columns	Informative	Redundant*	Noise (class.)	Noise (regr.)
50	8	8	34 (68%)	42 (84%)
100	10	10	80 (80%)	90 (90%)
500	50	50	400 (80%)	450 (90%)
1,000	100	100	800 (80%)	900 (90%)
5,000	300 (capped)	300	4,400 (88%)	4,700 (94%)

* Redundant = linear combinations of informative columns (classification only, via `n_redundant`). Regression has no equivalent — its “noise” column count is simply everything past the informative count.

Models compared

- **TabPFN-2.5** — Nov 2025 release; design ceiling 100K rows × 2,000 features.
- **TabPFN-3** — May 2026 release; row/feature trade-off, default regime favors more rows over more columns.
- **TabICL v2** — pretrained on 2–100 columns, claims but does not guarantee generalization beyond that.
- Both TabPFN variants run with `ignore_pretraining_limits=True` — the whole point of a failure-case study is to push past each model's stated comfort zone and observe what actually happens, not to stay safely inside it.

`run_benchmark.py` sweeps task × column-count × seed × model, recording accuracy/ROC-AUC (classification) or R^2 /RMSE (regression) plus fit and predict wall-clock time, writing incrementally to `results/results.csv`. `plot_results.py` turns that into line graphs.

3. Main Study — Results

Full single-seed sweep complete (seed=0, 100 train rows, 500 test rows, 7 column counts × 3 models × 2 tasks = 42 runs, all status=ok). Two distinct failure modes showed up, not one.

3.1 Accuracy failure — regression, 100–500 columns

TabPFN-2.5 is clearly the most robust: $R^2 = 0.956$ at 200 columns vs. 0.663 (TabPFN-3) and 0.538 (TabICL v2) — the strongest single piece of evidence for the original hypothesis. By 1,000 columns all three collapse to near-zero R^2 regardless of architecture — there's no signal left to preserve, so the difference stops mattering. TabICL v2 is the only model to go negative ($R^2 = -0.088$ at 2,000 columns), a qualitatively worse failure than “no signal” — its predictions are actively anti-correlated with the target.

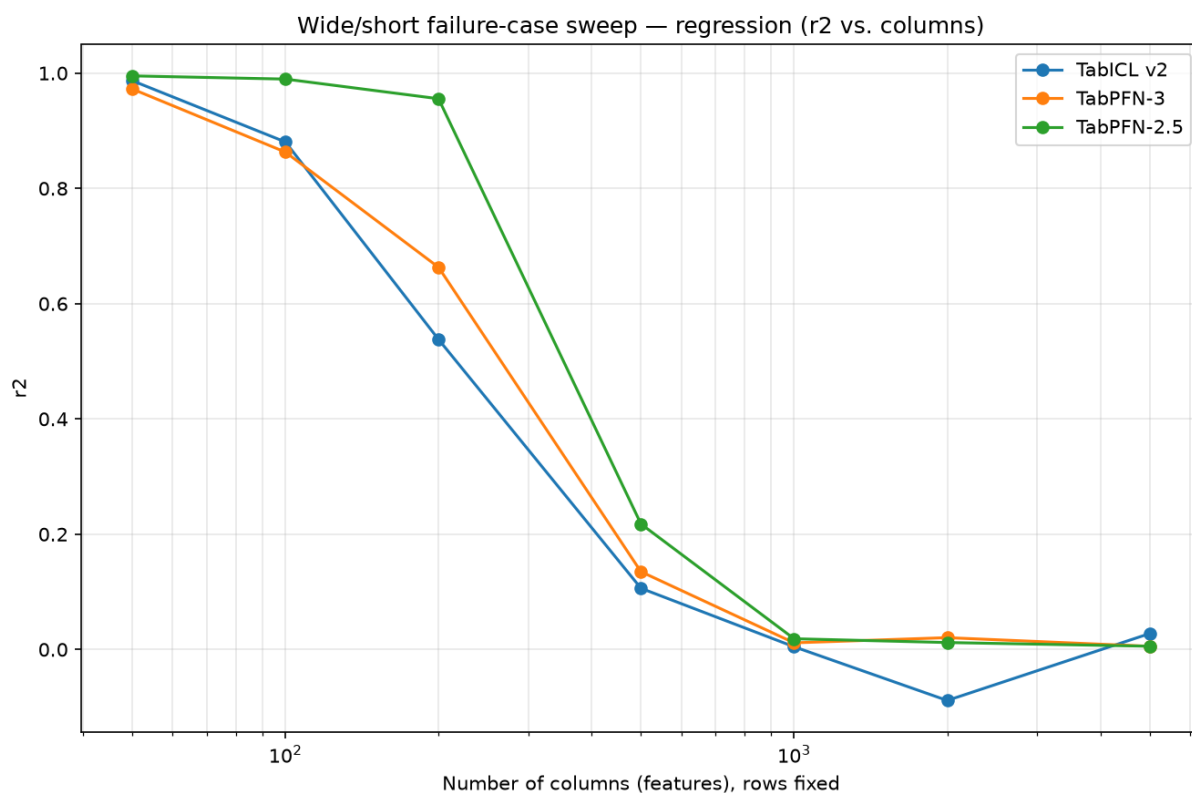


Figure 1 — Regression R^2 vs. column count, all three models, seed=0.

3.2 Accuracy in classification is messier

All three models track closely through 200 columns (~ 0.92 – 0.98 accuracy) and degrade together up to 1,000 columns. Past that point they diverge: TabICL v2 ends up the *best* performer at 5,000 columns (0.61), while TabPFN-3 — the newest model — becomes the *worst* (0.524), a full reversal from being tied-best at low column counts. TabPFN-2.5 sits in between (0.554). The “newer architecture degrades relatively worse” pattern holds for TabPFN-3 but not for TabICL in this task.

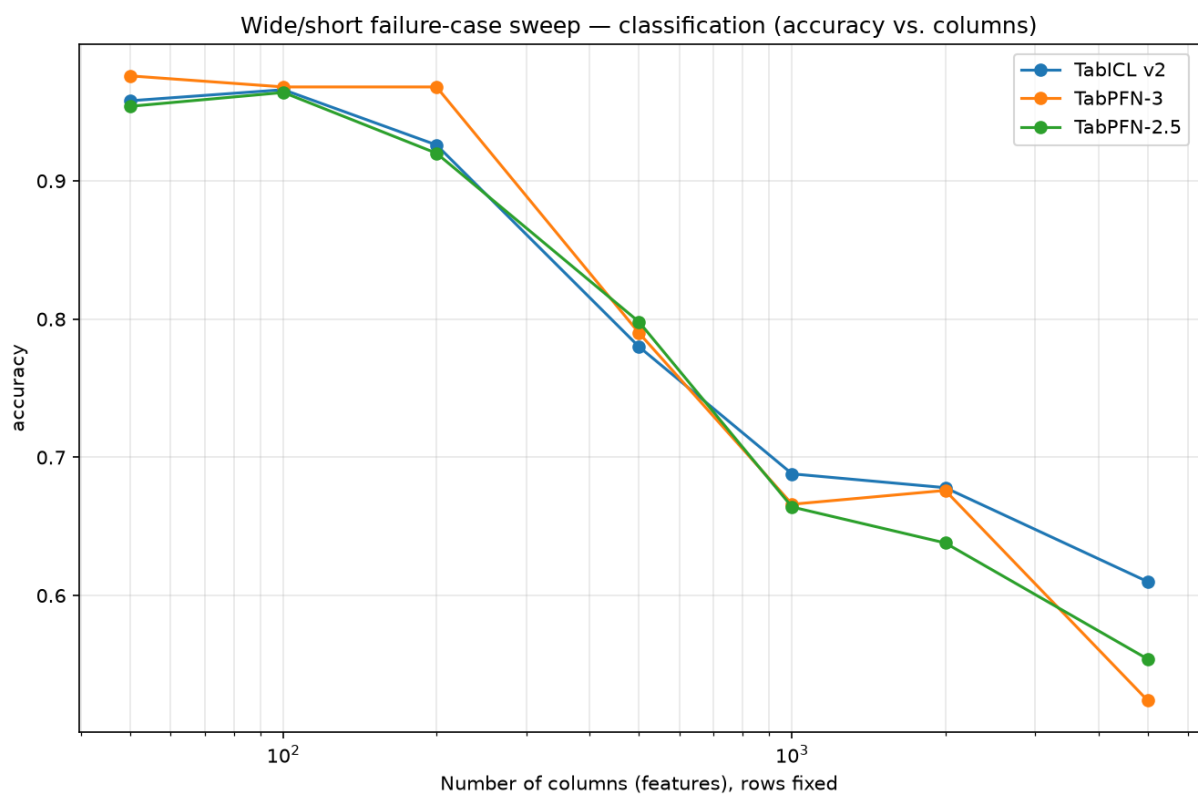


Figure 2 — Classification accuracy vs. column count, all three models, seed=0.

3.3 Computational failure — predict time, all tasks, high columns

The cleanest and most dramatic finding. Through 200 columns, predict time is close across all three models. Past that, TabICL v2's predict time explodes non-linearly while both TabPFN variants stay comparatively flat: at 5,000 columns, TabICL v2 takes ~1,776s (~30 min) to predict on 500 test rows, vs. 45.8s (TabPFN-2.5) and 18.2s (TabPFN-3) — 40–100x longer. This lines up with the architecture story above: TabPFN's per-estimator feature subsampling structurally caps how many columns any single ensemble member processes, bounding its cost; TabICL v2 has no equivalent cap, so its $O(n^2 + nm^2)$ column-attention cost scales directly with total feature count. TabICL v2's accuracy at this same extreme (0.61) was actually competitive with the other two models — its failure mode isn't wrong answers, it's becoming computationally impractical.

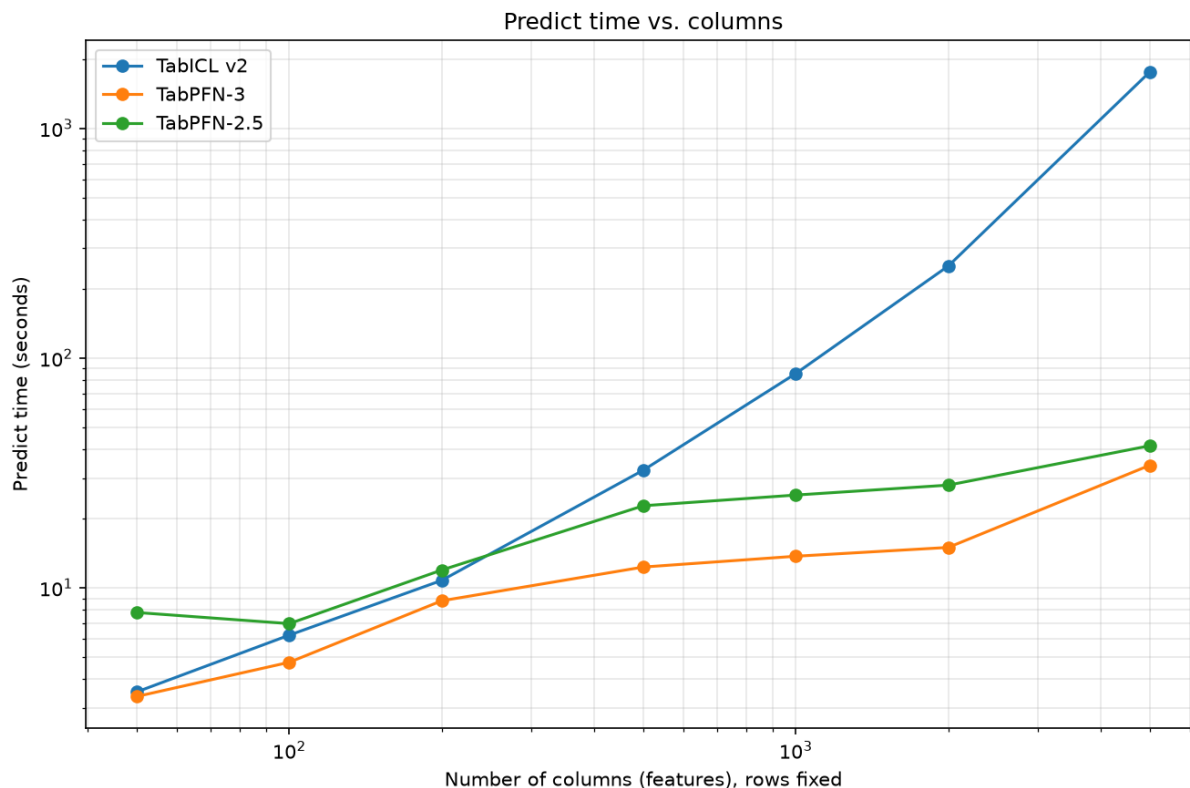


Figure 3 — Predict wall-clock time vs. column count (log scale), all three models, seed=0.

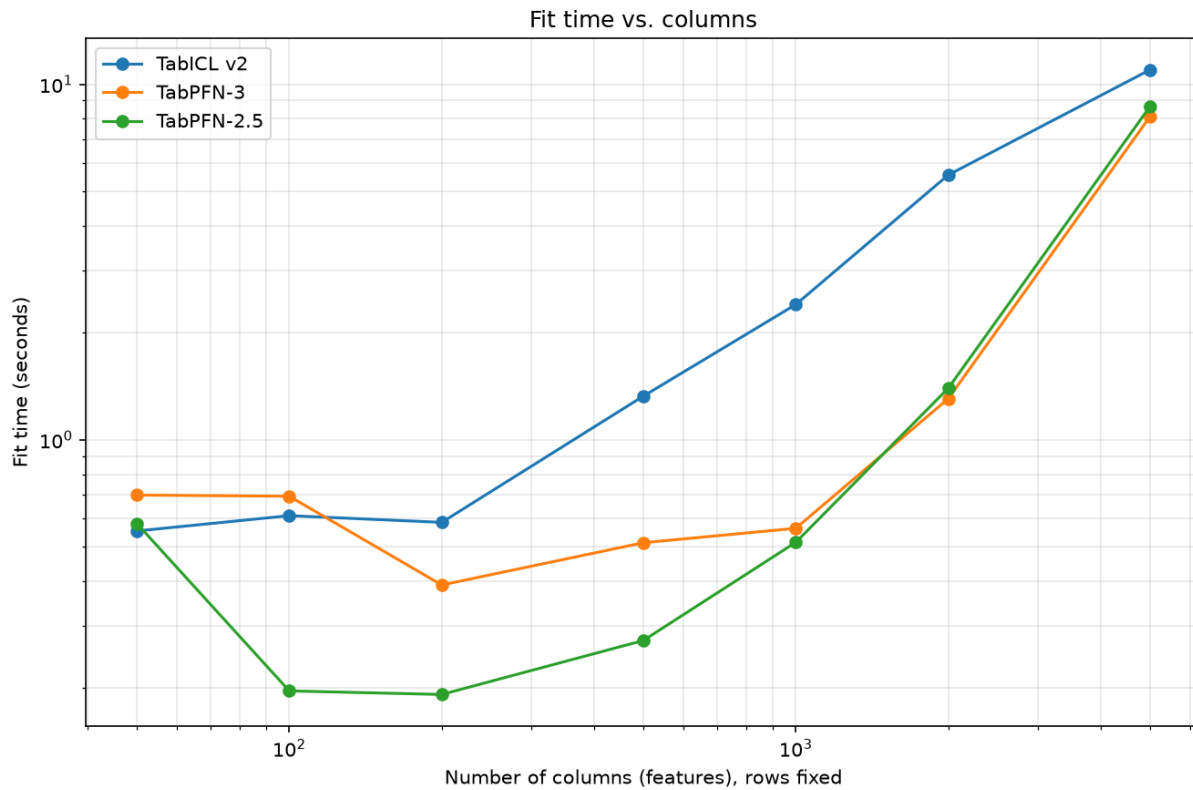


Figure 4 — Fit wall-clock time vs. column count, all three models, seed=0.

3.4 Limitations

- **Single seed only.** All results above come from seed=0, one full pass through the sweep — no second or third seed has been run. The regression robustness gap and the predict-time blowup are large enough relative to expected seed noise to be trustworthy as-is; the exact classification crossover point (e.g. where TabICL overtakes TabPFN-3) should be treated as illustrative rather than proven until confirmed across multiple seeds.
- **Ratio confound (open, not yet resolved).** Rows are fixed at 100 for the entire column sweep, so “columns increasing” and “feature/row ratio increasing” are never separated. Whether harder cases are “harder because there’s more noise to sift through” or “harder because 100 rows can’t estimate relationships across this many true dimensions, regardless of noise” can’t currently be distinguished. A rows × columns grid would resolve this — proposed, not yet built.
- **No feature-attribution pass.** Whether the models are actually attending to informative columns rather than latching onto noise coincidentally would need SHAP or similar — deliberately dropped from scope for this project (see §6).

4. Second Study — Arithmetic-Chain Generator

Two gaps were identified between the main study and the assignment brief: the brief explicitly asks for out-of-distribution (OOD) testing and for a comparison of “hierarchical decision complexity” (worked example: 20 steps vs. 5 steps). The main study's sklearn-based generator covers neither — its labeling rule is always a single linear cut or weighted sum, one step regardless of column count, and train/test are always drawn from the same distribution. A second, structurally different generator was built to close both gaps at once.

4.1 How the generator works

`arithmetic_generation.py` builds labels by chaining depth sequential arithmetic operations (+, -, *, /, mean) over a fixed set of active columns, instead of one linear rule. `depth=5` applies 5 chained operations; `depth=20` applies 20 — this directly maps onto the brief's hierarchical decision-complexity comparison. The active columns are always the first `n_active` (default 20) columns, fixed regardless of total width, and built from a “structure” RNG seeded only from `seed` — so the exact same rule is reused at every point in a column sweep, and only the amount of surrounding noise changes as width grows.

Concrete example — the `depth=5` chain used in Experiment 1 (`seed=0`):

```
result = x12
result = result + x3
result = result / x3
result = result - x19
result = result * x4
result = result + x12
```

Active columns are always `x0–x19` (20 columns); everything from `x20` onward is pure noise, regardless of total width. Classification labels are a train-median threshold split on the chain's output; regression labels are the chain's output plus Gaussian noise scaled to 10% of its train-set standard deviation.

The generator also supports a genuine train/test distribution shift (`ood=True`): test features are drawn from a shifted/rescaled normal distribution (mean shift +2, scale $\times 1.5$) instead of matching train — a true OOD test, unlike the main study where train and test always come from the same distribution.

4.2 Two bugs found and fixed during development

- **Column-count confound.** An early version drew active columns with `rng.choice(n_features, ...)`, whose output depends on population size even at a fixed seed — so a column sweep was silently testing a different random rule at every width instead of the same rule diluted by more noise. This produced a non-monotonic accuracy bounce (recovering at 2,000 columns after collapsing at 100) that all three models tracked in lockstep — the key diagnostic clue, since a real architectural effect should make models diverge, not move together. Fixed by splitting into a fixed-seed “structure” RNG (drives active columns and the chain, independent of `n_features`) and a separate “data” RNG (drives feature values and label noise).
- **Regression-target outlier bug.** The chain's `/` operation only floors the denominator at `1e-3`, so a single near-zero draw could spike the result by up to $\sim 1000\times$. A handful of these outliers then dominated the regression target — and swamped R^2 — at every column count, not just at high columns, making the whole regression curve look uniformly broken rather than showing any real width-driven trend. Fixed by robustly clipping the chain's output ($\text{median} \pm 10 \times \text{MAD}$, computed from train and applied to both splits) before it becomes a label. Classification was unaffected (its threshold is the train median, which a symmetric clip doesn't move).

5. Arithmetic-Chain — Experiment 1 (base) Results

Column sweep 50 → 2,000 (matching the main study's revised range — see §7), fixed depth=5, in-distribution, all 4 models including the XGBoost baseline (§6). Complete and verified after both bugs above were fixed.

5.1 Classification

Columns	TabPFN-2.5	TabPFN-3	TabICL v2	XGBoost
50	0.710	0.710	0.692	0.654
100	0.712	0.708	0.612	0.682
200	0.748	0.742	0.556	0.632
500	0.648	0.682	0.554	0.628
1,000	0.606	0.628	0.548	0.622
2,000	0.574	0.570	0.492	0.550

All four models decline as columns grow — roughly mid-70s% down to high-40s/low-50s% — but gradually, not as a sharp cliff. Unlike the main study, the decline doesn't clearly separate TabPFN (structurally protected) from TabICL (collapsing) — all four models decline together at broadly similar rates, ending within about 8 points of each other at 2,000 columns. This is a real, if less dramatic, finding: this task structure doesn't reproduce the main study's TabPFN-vs-TabICL divergence.

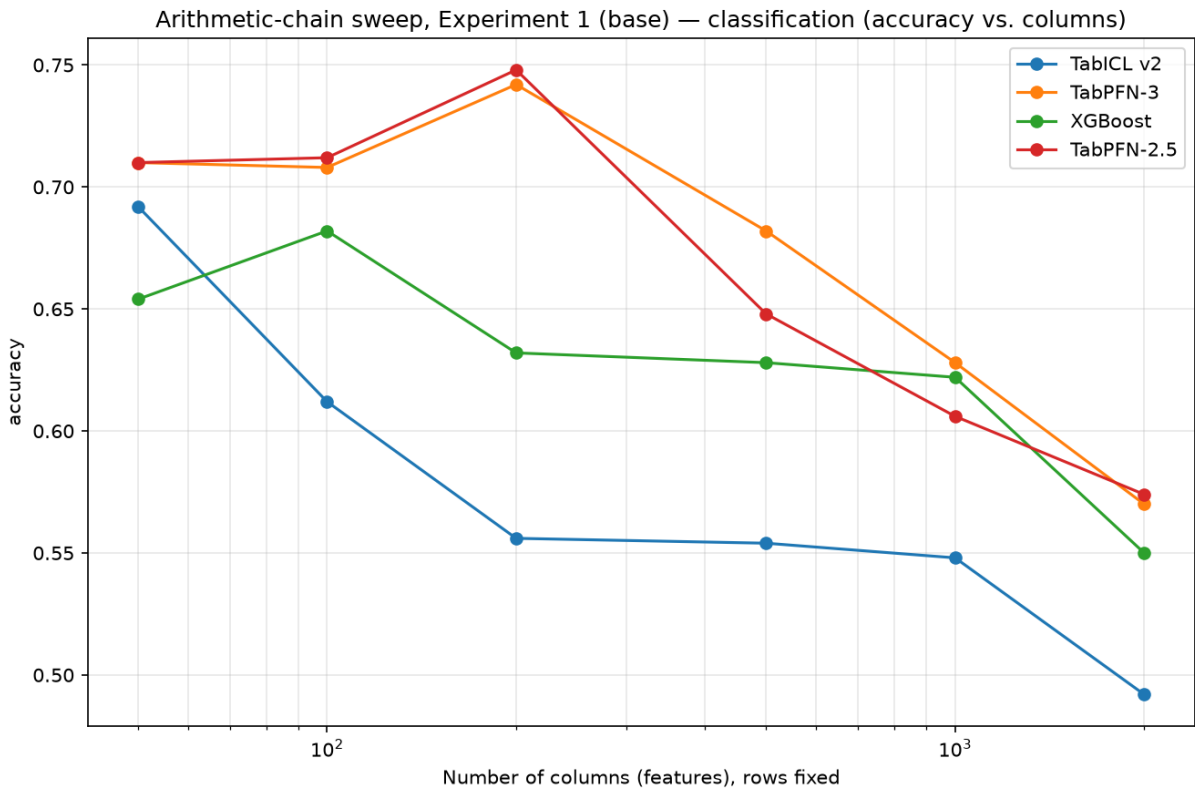


Figure 5 — Arithmetic-chain classification accuracy vs. columns, Experiment 1, all 4 models.

5.2 Regression

Columns	TabPFN-2.5	TabPFN-3	TabICL v2	XGBoost
50	0.046	0.050	-0.049	0.004
100	0.050	0.113	-0.015	-0.063
200	0.045	0.054	-0.357	-0.139
500	-0.016	-0.022	-0.007	-0.113
1,000	0.019	0.003	-0.108	-0.111
2,000	-0.011	0.009	-0.068	-0.105

R^2 values (post-fix) shown here.

R^2 stays close to zero for every model across the whole sweep, with no strong width-driven trend for any of them. This is a materially different, and much quieter, result than the main study's regression collapse: after fixing the outlier bug, this task's regression signal is simply weak throughout, for every model, at every column count — a legitimate result, just not a “failure case” in the same dramatic sense as §3.1.

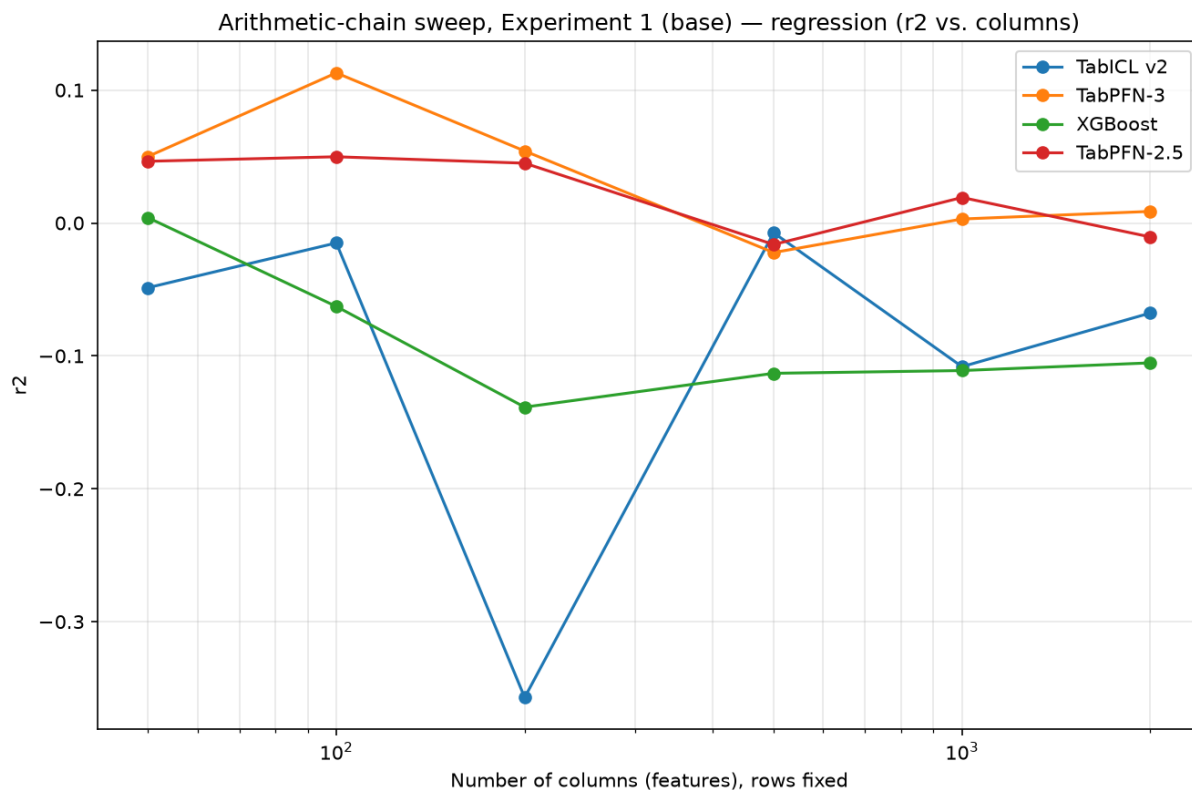


Figure 6 — Arithmetic-chain regression R^2 vs. columns, Experiment 1, all 4 models.

6. XGBoost Baseline

Added to give the poster a legible “TFM vs. classical ML” comparison — a way to show “TFM accuracy dropped” against “and here's what a non-TFM does in the same regime,” rather than only comparing the TFMs against each other. Included in all pipelines going forward by design (model-agnostic architecture — no runner or plotting code needed to change to add it).

Hyperparameters — deliberately regularized, not left at library defaults

- `max_depth=4, n_estimators=200` — shallow trees, small ensemble. With only 100 training rows and up to thousands of columns, an untuned/deep XGBoost would just overfit and take the comparison out of good faith.
- `colsample_bytree=0.5` — each tree sees only half the columns, a direct (if cruder) analogue to TabPFN's per-estimator feature subsampling, so the baseline isn't compared on “sees everything vs. sees a slice” terms.
- Runs entirely on CPU, regardless of the TabPFN/TabICL device settings.

Result so far (Experiment 1, arithmetic study)

On classification, XGBoost ends up closer to TabICL's collapse than to TabPFN's (0.550 vs. TabPFN's ~0.57 and TabICL's 0.492 at 2,000 columns). On regression, XGBoost is the worst performer of the four — consistently negative R^2 from 100 columns onward (down to -0.14), vs. the TFMs staying near zero — even with the regularization above, XGBoost's trees appear to overfit once column count meaningfully outpaces the 100 training rows.

Not yet run against the main study. XGBoost has only been benchmarked in the arithmetic-chain study so far; `results/results.csv` (the main study's results file) still reflects the original 3-model sweep. Re-running the main study with XGBoost included is straightforward but not yet done.

7. Coverage vs. the Assignment Brief

Brief requirement	Status	Where
Wide/short synthetic data, multiple TFMs	Done	Main study, §2–3
Classical-ML baseline for comparison	Done	XGBoost, §6
Out-of-distribution (OOD) testing	Generator built; sweep not yet run	Arithmetic study, Experiment 2
Hierarchical decision complexity (5 vs. 20 steps)	Generator built; sweep not yet run	Arithmetic study, Experiment 3
Feature attribution (SHAP)	Explicitly dropped from scope	§8
Multi-seed statistical confidence	Explicitly dropped from scope	§8

8. Current Status and Next Steps

8.1 Complete

- Main study: full single-seed sweep, 50–5,000 columns, 3 models, both tasks (42 runs).

- Arithmetic-chain study, Experiment 1 (base): 50–2,000 columns, depth=5, all 4 models including XGBoost, both tasks, in-distribution, both known bugs fixed and verified.

8.2 Defined, not yet run

- **Experiment 2 (OOD):** same column sweep, each cell run twice — in-distribution and OOD (shifted test set) — so accuracy/ R^2 can be compared side by side within the same column count. Roughly 2x Experiment 1's runtime.
- **Experiment 3 (depth):** fixed column count (default 500), depth swept over [5, 10, 20] — the direct “5 steps vs. 20 steps” hierarchical-complexity comparison from the brief.
- Both will use the same tabpfn/tabicl-then-xgboost split-run pattern established for Experiment 1 (see §9 for why).

8.3 Explicitly out of scope

- **SHAP / feature attribution** — dropped; seed=0's results are being treated as notable enough to report directly.
- **Multi-seed runs** — dropped for the same reason; all results in this document are single-seed (seed=0).

8.4 Identified but not yet built

- **Rows × columns grid** for the main study, to separate “harder because more noise” from “harder because too few rows for the true dimensionality” (the ratio confound, §3.4). Proposed, not started.
- **Main study re-run with XGBoost** included, to bring the classical-ML baseline into the same comparison as the TFM-vs-TFM main results.

9. Appendix — Reproducing This Work

9.1 Files

- `data_generation.py` — synthetic wide/short dataset generator (main study).
- `arithmetic_generation.py` — arithmetic-chain dataset generator (second study).
- `models.py` — unified wrappers for all 4 models (TabPFN-2.5, TabPFN-3, TabICL v2, XGBoost).
- `run_benchmark.py` / `run_benchmark_arithmetic.py` — sweep runners for each study.
- `plot_results.py` / `plot_arithmetic_results.py` — turn each study's CSV into line graphs.
- `SETUP.md` — environment setup, device notes, Hugging Face license step.
- `results/` — both studies' CSVs and all plots.

9.2 A technical note on running XGBoost alongside TabPFN/TabICL

XGBoost and torch (used by TabPFN/TabICL for MPS acceleration) each bundle their own OpenMP runtime; loading both into one process and then doing real MPS compute work reliably segfaults on macOS, with no Python-catchable exception. Device detection in `models.py` is lazy specifically so a process that only ever asks for XGBoost never imports torch at all. In practice this means TabPFN/TabICL and XGBoost are run as two separate invocations that append to the same results CSV, rather than one combined run:

```
python3 run_benchmark_arithmetic.py --models tabpfn_v2_5 tabpfn_v3 tabicl_v2
python3 run_benchmark_arithmetic.py --models xgboost
python3 plot_arithmetic_results.py
```

9.3 Running Experiments 2 and 3

```
python3 run_benchmark_arithmetic.py --experiment 2 --models tabpfn_v2_5 tabpfn_v3 tabicl_v2
python3 run_benchmark_arithmetic.py --experiment 2 --models xgboost
python3 run_benchmark_arithmetic.py --experiment 3 --models tabpfn_v2_5 tabpfn_v3 tabicl_v2
python3 run_benchmark_arithmetic.py --experiment 3 --models xgboost
python3 plot_arithmetic_results.py
```